# Software Design Document

for

# EduAssist-AI

Learning Companion System

## Version 1.0

**Prepared by:**

Sri Satya Sai Immani (Team Leader)
Pritesh Gurjar
Alex Christian

Case Western Reserve University
Department of Computer Science

October 6, 2025

# Revision History

| Date | Version | Description | Author |
|---|---|---|---|
| 2025-10-01 | 0.1 | Initial architecture draft | Team |
| 2025-10-06 | 1.0 | Final review and formatting | Team |

# Contents

# 1 Introduction

## 1.1 Purpose

This Software Design Document (SDD) provides a comprehensive architectural overview of the EduAssist-AI Learning Companion System using the IEEE 1016-2009 standard and the 4+1 architectural view model.

**Intended Audience:**

- **Developers**: System structure and implementation guidance
- **Architects**: Design decisions and patterns
- **Testers**: Integration points and test strategies
- **Stakeholders**: Technical scope validation

## 1.2 Scope

EduAssist-AI is a course room-based learning management system that transforms lecture videos into interactive study materials using AI-powered content generation.

**Core Capabilities:**

- **Faculty**: Create courses, upload videos, generate AI content, publish materials
- **Students**: Join courses, access materials, view transcripts, complete assessments

## 1.3 Definitions and Acronyms

| | |
|---|---|
| **API** | Application Programming Interface |
| **ASR** | Automatic Speech Recognition |
| **CLIP** | Contrastive Language-Image Pre-training (OpenAI vision model) |
| **FERPA** | Family Educational Rights and Privacy Act |
| **JWT** | JSON Web Token |
| **LLM** | Large Language Model |
| **RAG** | Retrieval-Augmented Generation |
| **RBAC** | Role-Based Access Control |
| **REST** | Representational State Transfer |
| **SDD** | Software Design Document |
| **SRS** | Software Requirements Specification |
| **TLS** | Transport Layer Security |

## 1.4 References

1. EduAssist-AI Software Requirements Specification v1.0 (September 22, 2025)
2. IEEE Std 1016-2009: Software Design Descriptions
3. Kruchten, P., "The 4+1 View Model of Software Architecture" (1995)
4. FastAPI Documentation: https://fastapi.tiangolo.com
5. MongoDB Documentation: https://docs.mongodb.com
6. HuggingFace Transformers Documentation: https://huggingface.co/docs

# 2 Architectural Representation

This document uses the 4+1 architectural view model to present multiple perspectives of the system architecture.

## 2.1 The 4+1 Views

| View | Audience | Purpose |
|---|---|---|
| Logical View | Designers, Architects | Functional decomposition and object model |
| Process View | Integrators, Performance Engineers | Concurrency and distribution |
| Implementation View | Programmers | Software organization and modules |
| Deployment View | System Engineers, DevOps | Physical distribution |
| Use Case View | All Stakeholders | Key scenarios and validation |
| Data View | Database Administrators | Data structures and persistence |

Table 1: Architectural views and their purposes

# 3 Architectural Goals and Constraints

## 3.1 Technical Platform Constraints

**Decision:** Web-based deployment with containerization

**Technology Stack:**

- **Frontend**: React 18.2, Tailwind CSS 3.3
- **Backend**: Python 3.11, FastAPI 0.104, Uvicorn 0.23
- **Database**: MongoDB 7.0
- **Cache/Queue**: Redis 7.2
- **Task Queue**: Celery 5.3
- **Deployment**: Docker 24.0 containers

**Rationale:** Maximum accessibility without desktop installation requirements. MongoDB's document model aligns with nested data structures (videos containing transcripts, summaries, quizzes).

## 3.2 Multi-Tenancy and Data Isolation

**Constraint:** Strict course-level data isolation for FERPA compliance

**Implementation:**

- All database queries filtered by `course_id`
- Enrollment-based authorization checks
- No cross-course data visibility
- Middleware enforces access controls at every layer

## 3.3 Security Requirements

**Authentication:** JWT tokens with Bcrypt password hashing (cost factor: 12)

**Authorization:** Role-Based Access Control (RBAC): Faculty vs Student

**Data Protection:**

- TLS 1.3 for all communications
- Encrypted database connections
- 4-hour session timeout
- Comprehensive audit logging

## 3.4 Asynchronous Processing

**Constraint:** Video processing takes minutes and cannot block HTTP requests
**Solution:**

- Celery distributed task queue
- Redis message broker
- Background workers for long-running operations
- Email notifications on completion

## 3.5 Performance Requirements

| **Operation** | **Target** | **Maximum** |
|---|---|---|
| User login | $<$ 1s | 2s |
| Load course list | $<$ 500ms | 1s |
| Video processing | 5 min/hr | 10 min/hr |
| AI chat response | $<$ 3s | 5s |
| Quiz submission | $<$ 1s | 2s |
| Concurrent users | 50+ | 100+ |

Table 2: Performance targets

# 4 Use Case View

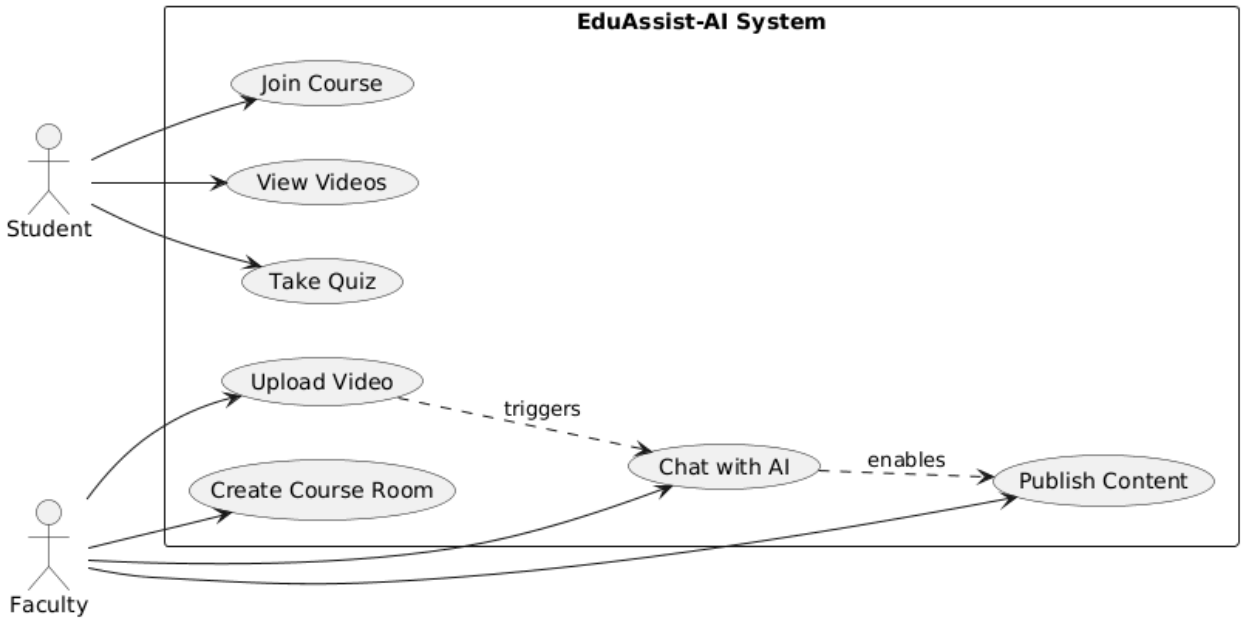


Figure 1: Use Case Overview Diagram

## 4.1 Architecturally Significant Use Cases

The following use cases drive major architectural decisions and require complex component interactions.

## 4.2 UC-1: Faculty Creates Course and Invites Students

**Priority:** High

**Description:** Faculty creates a course room and generates a unique invitation code for student enrollment.

**Architectural Impact:**

- Establishes multi-tenant data boundaries
- Requires collision-free invitation code generation
- Defines course ownership model

**Main Flow:**

1. Faculty provides course name and description
2. CourseService generates 8-character alphanumeric code

3. System creates course record with unique constraints
4. Faculty shares invitation code with students
5. Students join using code
6. System validates code and creates enrollment records

## 4.3 UC-2: Faculty Uploads and Processes Video

**Priority:** High

**Description:** Faculty uploads lecture video; system processes it asynchronously using ASR and scene detection.

**Architectural Impact:**

- Drives asynchronous processing architecture
- Requires external service integration
- Defines video state machine: PENDING → PROCESSING → COMPLETE → FAILED

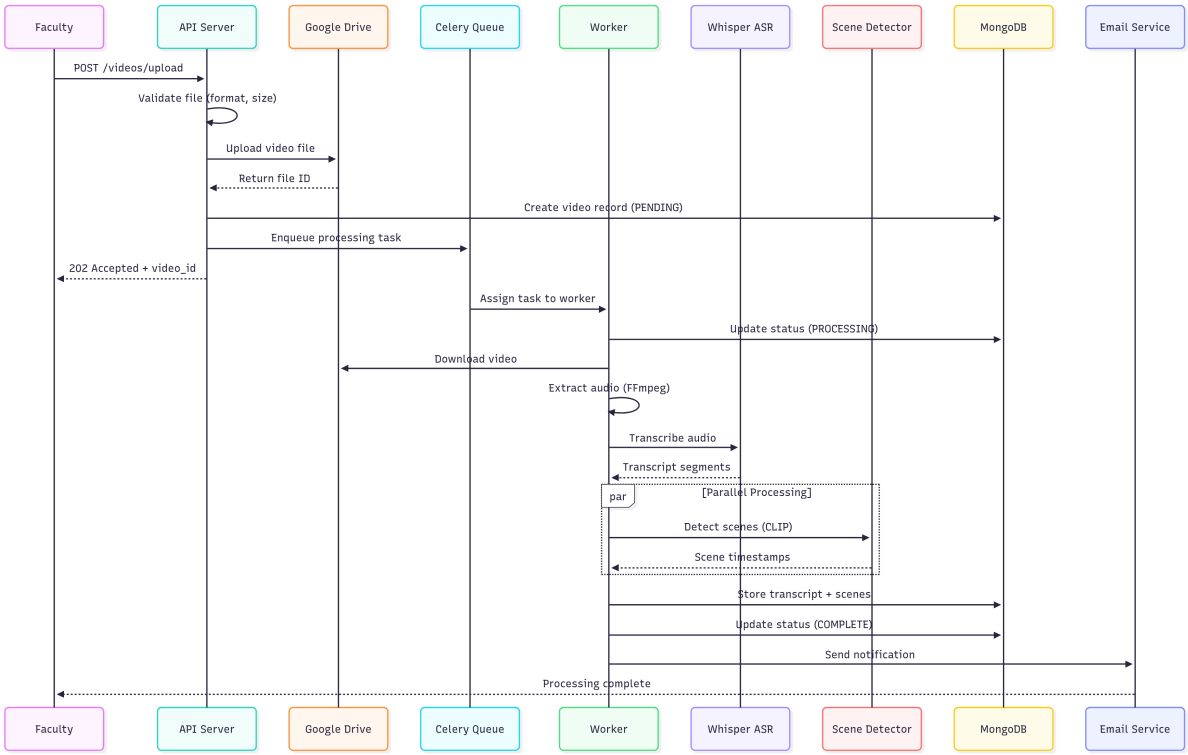


Figure 2: Video Upload and Processing Sequence Diagram

**Processing Pipeline:**

1. Upload video to Google Drive (max 2GB)
2. Create database record with status: PENDING
3. Enqueue background processing task
4. Worker extracts audio and transcribes with Whisper v3
5. Detect scenes using CLIP
6. Store transcript and update status: COMPLETE
7. Notify faculty via email

## 4.4 UC-3: Faculty Generates Content via AI Chat

**Priority:** High

**Description:** Faculty uses conversational AI to generate summaries, quizzes, and Q&A content.

**Conversational Flow:**

1. Faculty opens processed video in chat interface
2. System loads transcript and generates embeddings
3. Faculty: "Create a brief summary"
4. IntentParser identifies GENERATE_SUMMARY intent
5. SummarizationEngine retrieves context via RAG
6. LLM generates content, stored as DRAFT
7. Faculty refines: "Make it shorter"
8. Faculty publishes approved content

## 4.5 UC-4: Student Accesses Published Content

**Priority:** High

**Description:** Student joins course and views published materials.

**Security Enforcement:**

- All queries filtered by: enrolled courses + published=true
- Draft content invisible to students
- Cross-course access attempts logged and blocked

# 5 Logical View

## 5.1 Layered Architecture

The system employs an N-tier layered architecture with strict unidirectional dependencies.

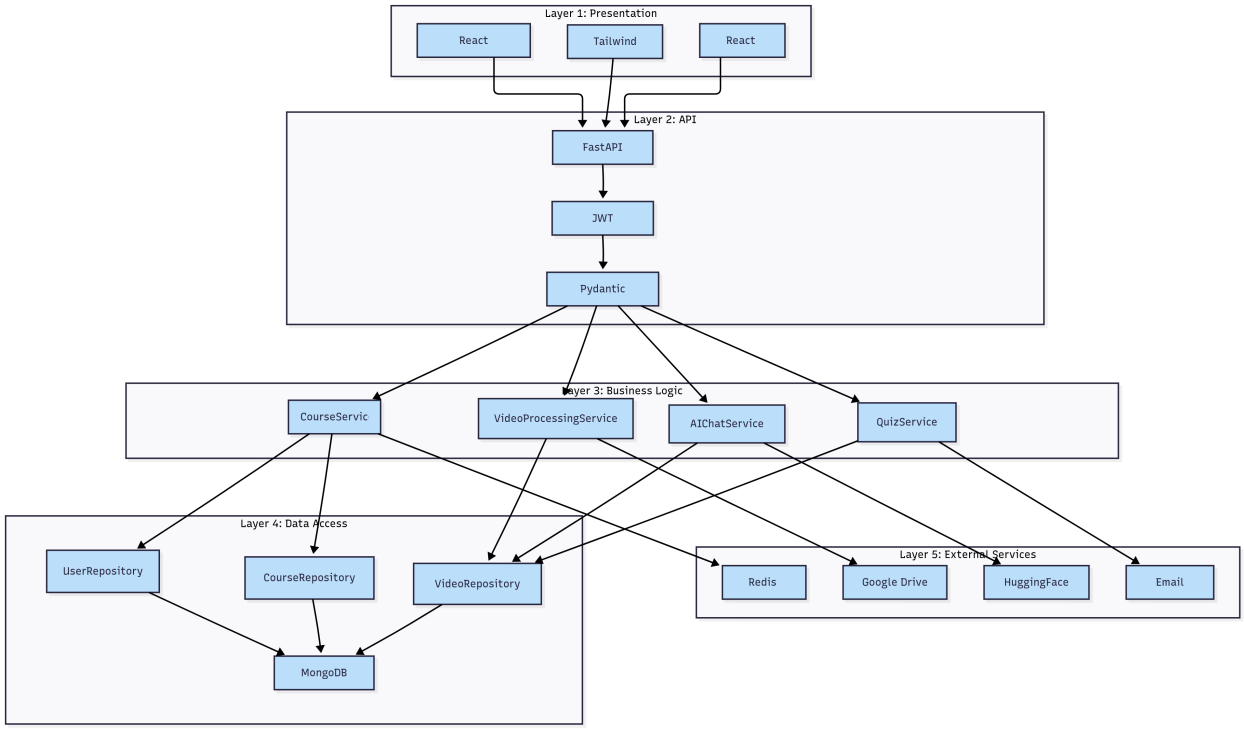


Figure 3: Five-Layer Architecture Diagram

### 5.1.1 Layer Responsibilities

**1. Presentation Layer (React)**

- UI rendering and client-side routing
- State management (React hooks)
- RESTful API communication via Axios

**2. API Layer (FastAPI)**

- Request handling and validation (Pydantic)
- JWT authentication middleware
- Response formatting and error handling

**3. Business Logic Layer**

- Core domain logic and business rules
- Workflow orchestration
- Service classes: CourseService, VideoProcessingService, AIChatService

**4. Data Access Layer**

- Repository pattern implementation
- Database queries and transactions
- Connection pooling

**5. External Services Layer**

- HuggingFace API integration
- Google Drive storage client
- Email notification service

## 5.2 Key Design Packages

### 5.2.1 Course Management Package

**Components:**

- `CourseService`: Course lifecycle management
- `InvitationService`: Unique code generation (excludes ambiguous characters: 0/O, 1/I/l)
- `EnrollmentService`: Student membership management
- `CourseRepository`: Data persistence

### 5.2.2 Video Processing Package

**Components:**

- `VideoUploadService`: Validation and cloud upload
- `VideoProcessingService`: ASR and scene detection orchestration
- `ProcessingQueue`: Celery task distribution
- `WhisperASRService`: Speech-to-text conversion (Whisper v3)
- `SceneDetectionService`: Visual analysis with CLIP

**Error Handling:**

- Automatic retry with exponential backoff (max 3 attempts)
- Transient failures trigger retry with 1s, 2s, 4s delays
- Permanent failures mark video as FAILED
- Detailed error logging with `video_id` and timestamps

### 5.2.3 Error Handling Strategy

**Exception Hierarchy:**

```
class AppException(Exception):
    """Base exception for all application errors"""
    pass

class ValidationException(AppException):
    """Invalid input parameters"""
    pass

class AuthenticationException(AppException):
    """Authentication failures"""
    pass

class BusinessLogicException(AppException):
    """Business rule violations"""
    pass

class ExternalServiceException(AppException):
    """Third-party service failures"""
    pass
```

Listing 1: Exception Class Hierarchy

**HTTP Status Code Mapping:**

| Status | Exception | Usage |
|---|---|---|
| 400 | ValidationException | Invalid input parameters |
| 401 | AuthenticationException | Missing or invalid JWT token |
| 403 | InsufficientPermissionsException | User lacks required permissions |
| 404 | NotFoundException | Resource does not exist |
| 409 | BusinessLogicException | Conflict (duplicate, already enrolled) |
| 413 | InvalidFileException | File too large (>2GB) |
| 415 | InvalidFileException | Unsupported file format |
| 429 | RateLimitException | Too many requests |
| 500 | InternalServerError | Unexpected system error |
| 502 | ExternalServiceException | External API failure |
| 503 | ServiceUnavailableException | System maintenance or overload |

Table 3: HTTP status codes and exception mapping

**Error Response Format**

```
{
  "error": {
    "code": "INVALID_INVITATION_CODE",
    "message": "The invitation code ABCD5678 is invalid or
       expired",
    "details": {
      "field": "invitationCode",
      "provided": "ABCD5678"
    },
    "timestamp": "2025-10-06T16:30:00Z",
    "requestId": "req_550e8400e29b41d4a716"
  }
}
```

**Retry Strategy for External Services:**

```
RetryPolicy = {
    "maxAttempts": 3,
    "backoffType": "EXPONENTIAL",
    "initialDelay": 1000,  # milliseconds
    "maxDelay": 10000,     # milliseconds
    "multiplier": 2.0,
    "retryableExceptions": [
        "NetworkTimeoutException",
        "TemporaryServiceException",
        "RateLimitException"
    ],
    "nonRetryableExceptions": [
        "AuthenticationException",
```

```
        "InvalidInputException"
    ]
}
```

Listing 2: Retry Policy Configuration

#### 5.2.4 AI Content Generation Package

**Components:**

- `FacultyAIChatService`: Chat session management
- `IntentParser`: Natural language understanding
- `SummarizationEngine`: Multi-length summary generation (FLAN-T5)
- `QuizGenerator`: MCQ and short-answer questions
- `RAGEngine`: Retrieval-augmented generation
- `EmbeddingService`: Vector representations (sentence-transformers)

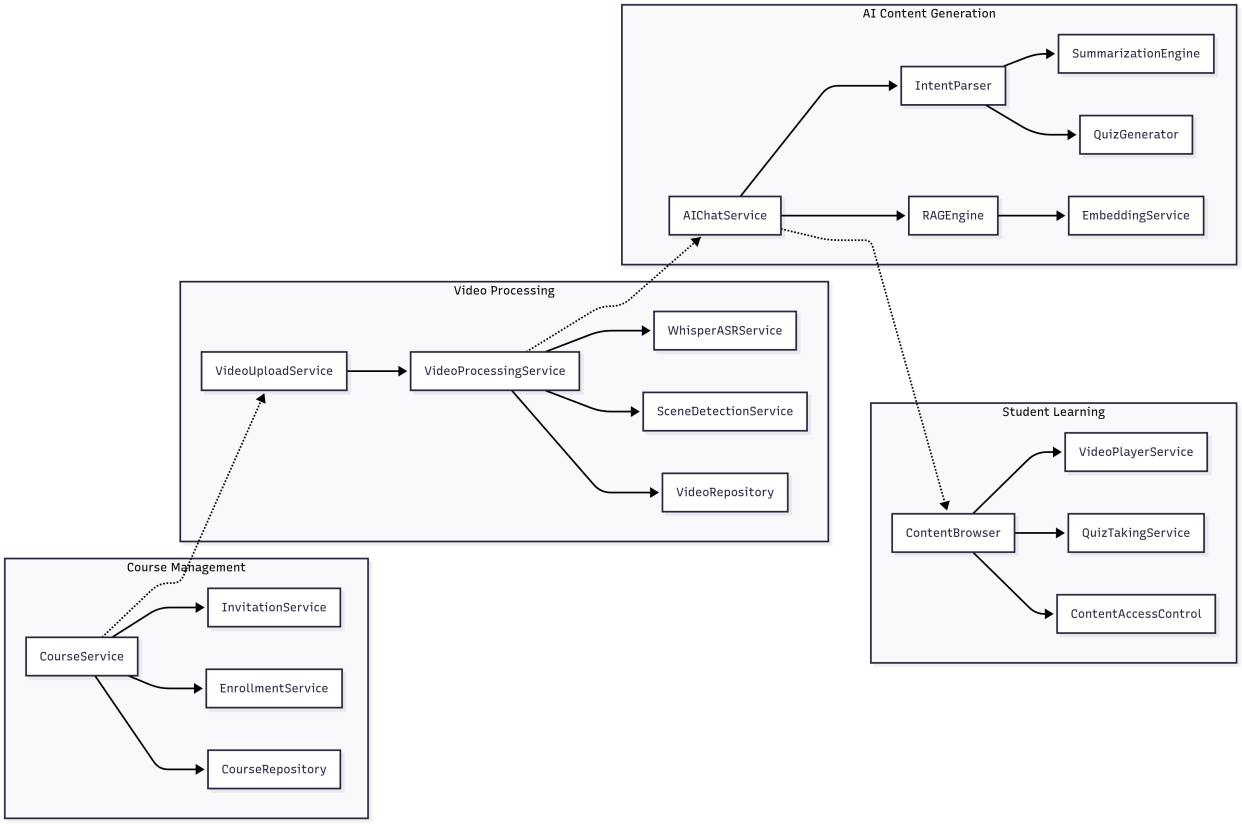


Figure 4: Component Interaction Diagram

## 5.3 Domain Model

**Core Entities:**

- **User**: Faculty and student accounts with role-based permissions
- **CourseRoom**: Container with unique 8-character invitation code
- **Enrollment**: User-course membership link with join timestamp
- **Video**: Lecture recording with processing status state machine
- **Transcript**: Time-aligned ASR output with word-level timestamps
- **Summary**: AI-generated content (BRIEF/DETAILED/COMPREHENSIVE)
- **Quiz**: Assessment with questions, answers, and explanations

**Aggregate Relationships:**

- CourseRoom is aggregate root for Videos and Enrollments
- Video is aggregate root for Transcript, Summaries, Quizzes
- Deletion cascades from aggregate root to children

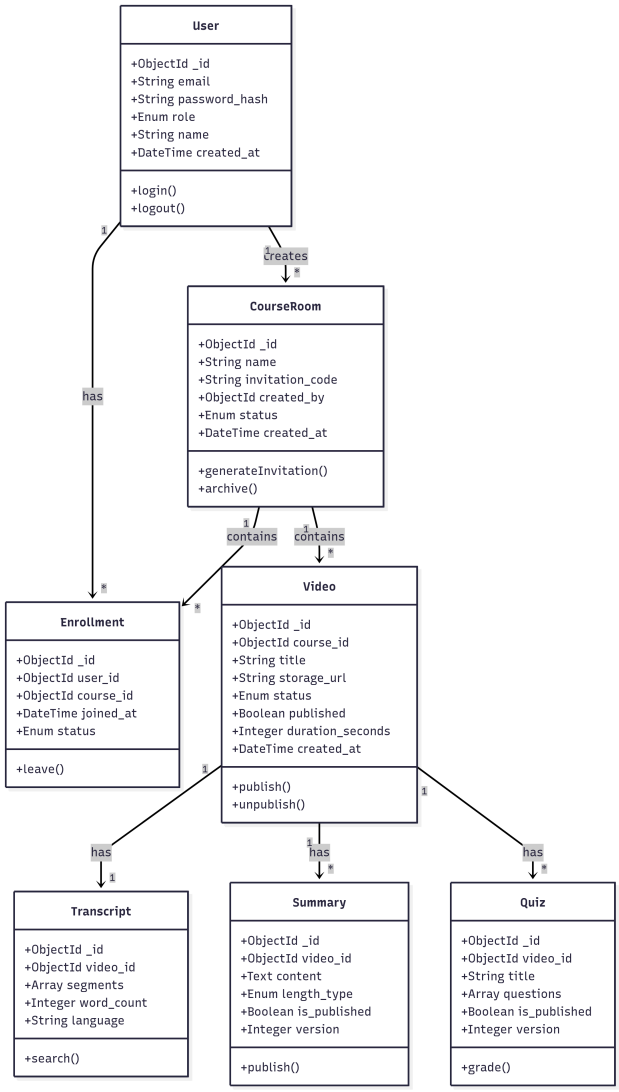


Figure 5: Core Domain Entities Class Diagram

## 5.4 Key Interface Specifications

### 5.4.1 ICourseService Interface

```python
from typing import Protocol
from bson import ObjectId

class ICourseService(Protocol):
    def create_course(
        self,
        user_id: ObjectId,
        name: str,
        description: str
```

```
    ) -$>$ CourseDTO:
        """
        Creates a new course with unique invitation code.

        Preconditions:
            - user_id must reference valid FACULTY user
            - name length: 3-100 characters
            - description length: 0-500 characters

        Postconditions:
            - Course record created in database
            - Unique 8-character invitation code generated
            - User added as course owner

        Raises:
            ValidationException: Invalid parameters
            DuplicateCodeException: Code collision (retry)
        """
        pass

    def join_course(
        self,
        user_id: ObjectId,
        invitation_code: str
    ) -$>$ EnrollmentDTO:
        """
        Enrolls student in course via invitation code.

        Preconditions:
            - user_id must reference valid STUDENT user
            - invitation_code must be 8 characters

        Postconditions:
            - Enrollment record created
            - User can access course content

        Raises:
            InvalidCodeException: Code not found
            AlreadyEnrolledException: Duplicate enrollment
        """
        pass
```

Listing 3: Course Service Interface

### 5.4.2 IVideoProcessingService Interface

```
class IVideoProcessingService(Protocol):
```

```
    def upload_video(
        self,
        user_id: ObjectId,
        course_id: ObjectId,
        file: FileUpload
    ) -$>$ VideoDTO:
        """
        Uploads video and queues for processing.

        Preconditions:
            - user_id must be course owner
            - file.size $<$= 2GB
            - file.mimetype in ['video/mp4', 'video/avi', 'video/mov
               ']

        Postconditions:
            - File uploaded to Google Drive
            - Video record created with status=PENDING
            - Background processing task enqueued

        Raises:
            FileSizeException: File exceeds 2GB
            UnsupportedFormatException: Invalid format
        """
        pass

    def get_processing_status(
        self,
        video_id: ObjectId
    ) -$>$ ProcessingStatusDTO:
        """
        Returns current processing status.

        Returns:
            {
                "status": "PENDING|PROCESSING|COMPLETE|FAILED",
                "progress": 0-100,
                "error": str | None,
                "estimatedTimeRemaining": int (seconds)
            }
        """
        pass
```

Listing 4: Video Processing Service Interface

### 5.4.3 IAIChatService Interface

```python
class IAIChatService(Protocol):
    def send_message(
        self,
        user_id: ObjectId,
        video_id: ObjectId,
        message: str
    ) -$>$ ChatResponseDTO:
        """
        Processes conversational AI request.

        Preconditions:
            - User must be course owner
            - Video status must be COMPLETE
            - message length: 1-1000 characters

        Returns:
            {
                "response": str,
                "intent": IntentType,
                "generatedContent": ContentDTO | None,
                "suggestions": List[str]
            }

        Raises:
            UnauthorizedException: Not course owner
            VideoNotReadyException: Video not processed
        """
        pass

    def generate_summary(
        self,
        user_id: ObjectId,
        video_id: ObjectId,
        length_type: str  # 'BRIEF'|'DETAILED'|'COMPREHENSIVE'
    ) -$>$ SummaryDTO:
        """
        Generates structured summary.

        Rate Limit: 10 requests per minute per user

        Postconditions:
            - Summary created with is_published=false
            - Version number incremented
        """
        pass
```

Listing 5: AI Chat Service Interface

# 6 Process View

## 6.1 Concurrency Model

### 6.1.1 Stateless API Servers

**Design:** Multiple FastAPI instances behind Nginx load balancer
**Characteristics:**

- No shared in-process state
- Session state stored in Redis (shared across servers)
- Horizontal scaling capability
- Round-robin load distribution

**Threading:** Uvicorn ASGI with async/await, non-blocking I/O

### 6.1.2 Asynchronous Task Processing

**Worker Pool:** Celery distributed tasks with Redis broker
**Task Types:**

1. `video_processing`: ASR, scene detection (10-30 min)
2. `generate_embeddings`: Vector creation (2-5 min)
3. `send_notification`: Email alerts ($<$ 1 sec)
4. `cleanup_expired_sessions`: Maintenance ($<$ 1 min)

### 6.1.3 Database Connection Pooling

**MongoDB Configuration:**

- Minimum pool: 10 connections (kept warm)
- Maximum pool: 50 connections
- Idle timeout: 30 seconds
- Automatic retry on transient failures (3 attempts)

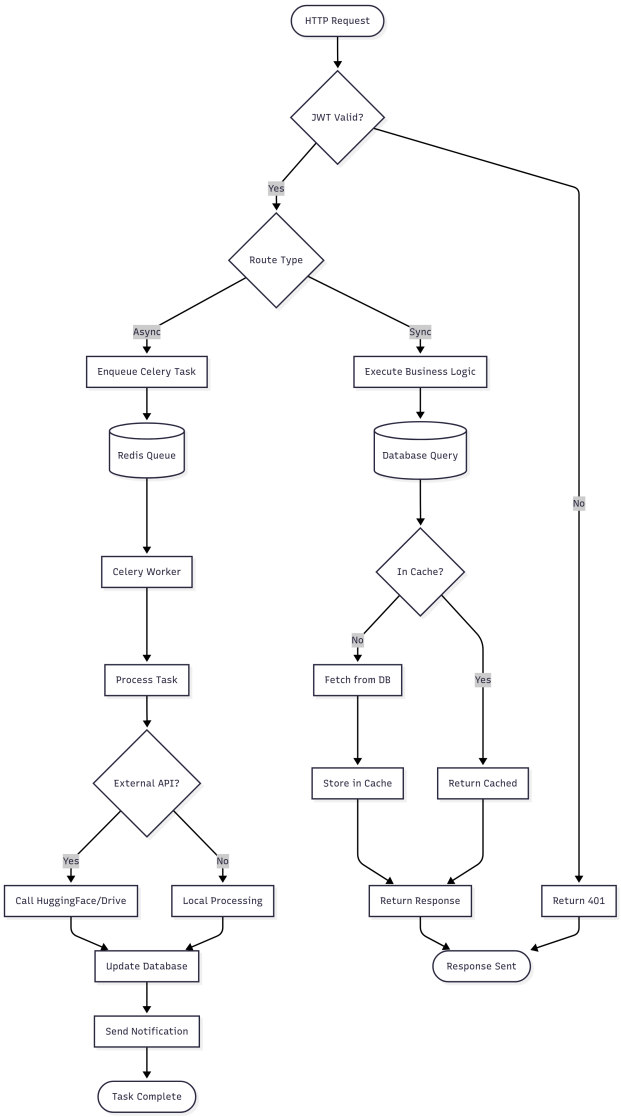


Figure 6: Concurrent Processing Activity Diagram

## 6.2 Synchronization Mechanisms

**Database Transactions:**

- Multi-document transactions for atomic operations
- Used for enrollment (check duplicate + insert)
- Used for quiz submission (attempt record + statistics update)

**Distributed Locks:**

- Redis-based locks for invitation code generation
- Prevents duplicate codes during concurrent creation

- 5-second lock timeout prevents deadlock
- Automatic lock release on process failure

# 7 Deployment View

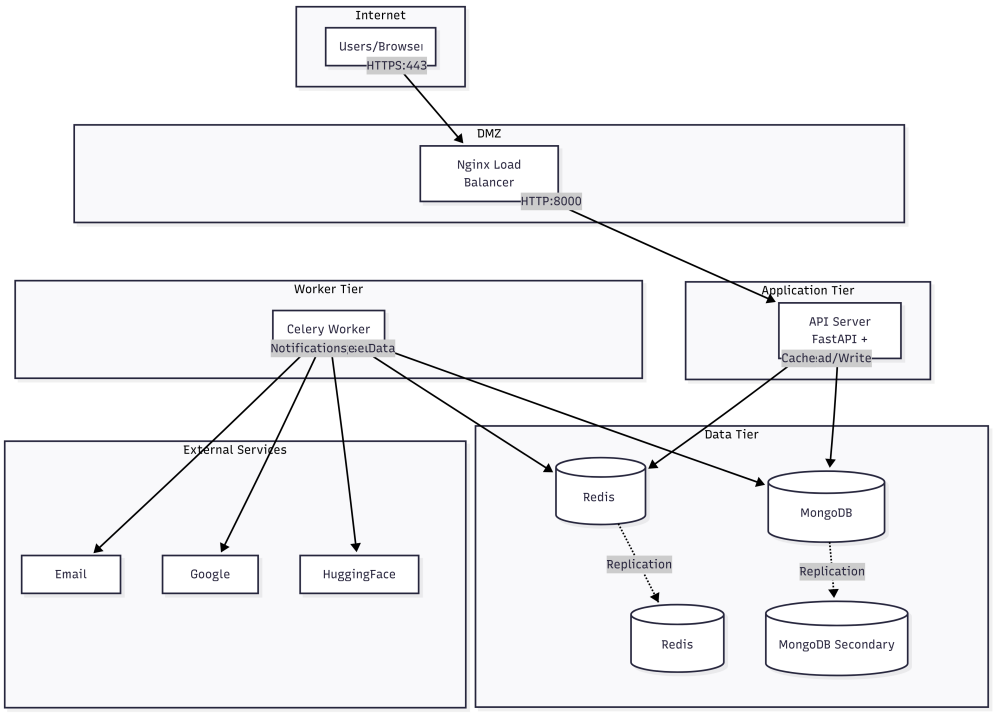


Figure 7: System Deployment Architecture

## 7.1 Node Specifications

### 7.1.1 Load Balancer (Nginx)

**Responsibilities:**

- SSL/TLS termination (Let's Encrypt certificates)
- Request distribution (round-robin with health checks)
- Static asset serving (CSS, JS, images)
- Rate limiting (100 req/min per IP)

### 7.1.2 API Server Nodes (3+ instances)

**Hardware per instance:**

- 2 vCPU cores
- 4 GB RAM
- 20 GB disk

- 100 Mbps network

**Scaling:** Auto-scale on CPU > 70%, min 2 / max 10 instances

#### 7.1.3 Celery Worker Nodes (2+ instances)

**Hardware per instance:**

- 4 vCPU cores
- 8 GB RAM
- 100 GB disk (temporary video files)
- 1 Gbps network

**Scaling:** Based on queue depth > 10 tasks

#### 7.1.4 MongoDB Replica Set

**Configuration:** 1 Primary + 2 Secondaries
**Hardware per node:**

- 4 vCPU cores
- 16 GB RAM
- 500 GB SSD

**Backup:** Daily full backups, 30-day retention, point-in-time recovery

#### 7.1.5 Redis Cluster

**Configuration:** 3-node cluster with replication
**Hardware:** 2 vCPU, 8 GB RAM, 20 GB disk

### 7.2 Network Topology

**External:** Load balancer public IP (443/HTTPS)
**Internal:** Private subnet (10.0.0.0/16), NAT gateway for egress
**Communication Paths:**

- Client → Load Balancer: HTTPS (443)
- Load Balancer → API Servers: HTTP (8000)
- API Servers → MongoDB: MongoDB Protocol (27017)
- API Servers → Redis: Redis Protocol (6379)
- Workers → External APIs: HTTPS (443)

# 8 Implementation View

## 8.1 Package Structure

```
eduassist/
          frontend/
                src/
                      components/      # Reusable UI components
                      pages/           # Page-level components
                      services/        # API client services
                      utils/           # Helper functions
                package.json

          backend/
                api/                 # FastAPI routers
                services/            # Business logic
                repositories/        # Data access
                models/              # Domain models
                external/            # External service clients
                workers/             # Celery tasks
                main.py

          tests/
                unit/
                integration/
                e2e/

          docker/
              Dockerfile.api
              Dockerfile.worker
              docker-compose.yml
```

Listing 6: Project Directory Structure

## 8.2 Layer Mapping

| Architecture Layer | Implementation Package |
|---|---|
| Presentation Layer | `frontend/` |
| API Layer | `backend/api/` |
| Business Logic Layer | `backend/services/` |
| Data Access Layer | `backend/repositories/` |
| External Services Layer | `backend/external/` |

Table 4: Layer to package mapping

## 8.3 REST API Endpoint Specifications

### 8.3.1 Authentication Endpoints

**POST /api/v1/auth/register**

**Request:**

```
{
  "email": "faculty@university.edu",
  "password": "SecurePass123!",
  "name": "Dr. Jane Smith",
  "role": "FACULTY"
}
```

**Response (201 Created):**

```
{
  "userId": "507f1f77bcf86cd799439011",
  "email": "faculty@university.edu",
  "role": "FACULTY",
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

**Errors:**

- 400: Invalid email format or password too weak
- 409: Email already registered

`POST /api/v1/auth/login`

**Request:**

```
{
  "email": "faculty@university.edu",
  "password": "SecurePass123!"
}
```

**Response (200 OK):**

```
{
  "userId": "507f1f77bcf86cd799439011",
  "email": "faculty@university.edu",
  "role": "FACULTY",
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "expiresAt": "2025-10-07T16:30:00Z"
}
```

**Errors:**

- 401: Invalid credentials
- 429: Too many login attempts (rate limit: 5/min)

### 8.3.2 Course Management Endpoints

`POST /api/v1/courses`

**Headers:** `Authorization: Bearer <token>`
**Request:**

```
{
  "name": "Neural Networks CS439",
  "description": "Deep learning fundamentals"
}
```

**Response (201 Created):**

```
{
  "courseId": "507f1f77bcf86cd799439012",
  "name": "Neural Networks CS439",
  "description": "Deep learning fundamentals",
  "invitationCode": "ABCD1234",
  "invitationLink": "https://eduassist.ai/join/ABCD1234",
  "createdAt": "2025-10-06T14:30:00Z",
  "status": "ACTIVE"
}
```

`POST /api/v1/courses/join`

**Headers:** `Authorization: Bearer <token>`
**Request:**

```
{
  "invitationCode": "ABCD1234"
}
```

**Response (200 OK):**

```
{
  "enrollmentId": "507f1f77bcf86cd799439013",
  "courseId": "507f1f77bcf86cd799439012",
  "courseName": "Neural Networks CS439",
  "role": "STUDENT",
  "joinedAt": "2025-10-06T15:00:00Z"
}
```

**Errors:**

- 404: Invalid invitation code
- 409: Already enrolled in course
- 403: Faculty users cannot join as students

### 8.3.3 Video Management Endpoints

**POST /api/v1/courses/{courseId}/videos**

**Headers:**

```
Authorization: Bearer $<$token$>$
Content-Type: multipart/form-data
```

**Request (Form Data):**

```
file: $<$video file binary$>$
title: "Lecture 1: Introduction to CNNs"
```

**Response (202 Accepted):**

```
{
  "videoId": "507f1f77bcf86cd799439014",
  "title": "Lecture 1: Introduction to CNNs",
  "status": "PENDING",
  "statusUrl": "/api/v1/videos/507f.../status",
  "estimatedProcessingTime": 300
}
```

**Errors:**

- 413: File size exceeds 2GB limit
- 415: Unsupported video format (only MP4, AVI, MOV)
- 403: User is not course owner

**GET /api/v1/videos/{videoId}/status**

**Response (200 OK):**

```
{
  "videoId": "507f1f77bcf86cd799439014",
  "status": "PROCESSING",
  "progress": 45,
  "currentStep": "Transcribing audio",
  "estimatedTimeRemaining": 180,
  "error": null
}
```

`GET /api/v1/courses/{courseId}/videos`

**Query Parameters:**

- `published`: boolean (optional, default: true for students)
- `status`: string (optional, filter by PENDING/PROCESSING/COMPLETE/-FAILED)
- `page`: integer (default: 1)
- `limit`: integer (default: 20, max: 100)

**Response (200 OK):**

```
{
  "videos": [
    {
      "videoId": "507f1f77bcf86cd799439014",
      "title": "Lecture 1: Introduction to CNNs",
      "durationSeconds": 3600,
      "status": "COMPLETE",
      "published": true,
      "publishedAt": "2025-10-05T10:00:00Z",
      "thumbnailUrl": "https://drive.google.com/...",
      "hasTranscript": true,
      "hasSummary": true,
      "hasQuiz": true
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 3,
    "totalItems": 45,
    "itemsPerPage": 20
  }
}
```

### 8.3.4 AI Content Generation Endpoints

`POST /api/v1/videos/{videoId}/chat`

**Request:**

```
{
  "message": "Create a brief summary of this lecture",
  "sessionId": "optional-session-id"
}
```

**Response (200 OK):**

```
{
  "response": "I've generated a brief summary focusing on...",
  "intent": "GENERATE_SUMMARY",
  "generatedContent": {
    "contentType": "SUMMARY",
    "contentId": "507f1f77bcf86cd799439015",
    "preview": "This lecture introduces CNNs...",
    "isPublished": false
  },
  "sessionId": "550e8400-e29b-41d4-a716",
  "suggestions": [
    "Make it more detailed",
    "Generate quiz questions",
    "Add key concepts"
  ]
}
```

### POST /api/v1/videos/{videoId}/summaries

**Request:**

```
{
  "lengthType": "DETAILED",
  "focusAreas": ["backpropagation", "activation functions"]
}
```

**Response (201 Created):**

```
{
  "summaryId": "507f1f77bcf86cd799439016",
  "videoId": "507f1f77bcf86cd799439014",
  "lengthType": "DETAILED",
  "content": "# Lecture Summary\n\n## Introduction...",
  "wordCount": 450,
  "version": 1,
  "isPublished": false,
  "createdAt": "2025-10-06T16:00:00Z"
}
```

### PATCH /api/v1/summaries/{summaryId}/publish

**Request:**

```
{
  "isPublished": true
}
```

**Response (200 OK):**

```
{
  "summaryId": "507f1f77bcf86cd799439016",
  "isPublished": true,
  "publishedAt": "2025-10-06T16:15:00Z",
  "version": 1
}
```

### 8.3.5 Quiz Endpoints

`GET /api/v1/videos/{videoId}/quizzes`

**Response (200 OK):**

```
{
  "quizzes": [
    {
      "quizId": "507f1f77bcf86cd799439017",
      "title": "Lecture 1 Quiz",
      "questionCount": 10,
      "isPublished": true,
      "version": 1,
      "averageScore": 85.5,
      "attemptCount": 23
    }
  ]
}
```

POST /api/v1/quizzes/{quizId}/attempts

**Request:**

```
{
  "answers": [
    {
      "questionIndex": 0,
      "answer": "2"
    },
    {
      "questionIndex": 1,
      "answer": "Backpropagation computes gradients..."
    }
  ]
}
```

**Response (201 Created):**

```
{
  "attemptId": "507f1f77bcf86cd799439018",
  "score": 80,
  "totalQuestions": 10,
  "correctAnswers": 8,
  "timeSpentSeconds": 420,
  "feedback": [
    {
      "questionIndex": 0,
      "isCorrect": true,
      "explanation": "Correct! CNNs use..."
    },
    {
      "questionIndex": 1,
      "isCorrect": false,
      "explanation": "Not quite. Backpropagation actually...",
      "correctAnswer": "Backpropagation is an algorithm..."
    }
  ]
}
```

# 9 Data View

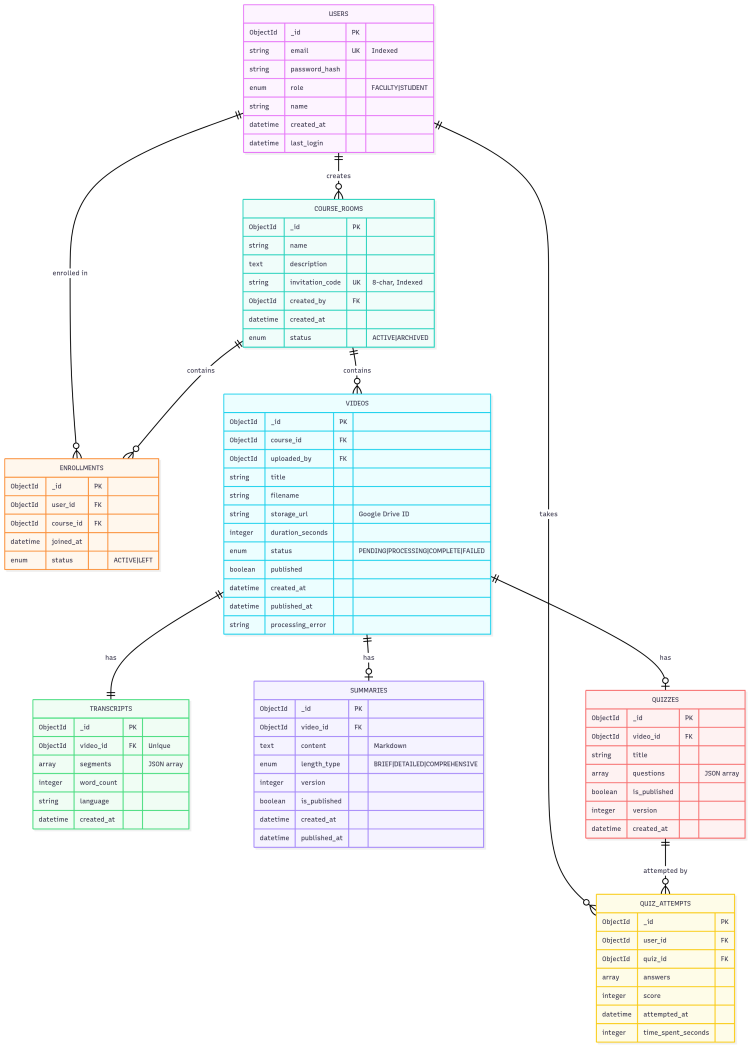


Figure 8: Entity-Relationship Diagram with Cardinalities

## 9.1 Core Collections

### 9.1.1 users Collection

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary key (auto-generated) |
| email | String | Unique email (indexed) |
| password_hash | String | Bcrypt hashed (cost factor: 12) |
| role | Enum | FACULTY or STUDENT |
| name | String | Display name (3-100 chars) |
| created_at | DateTime | Account creation timestamp |
| last_login | DateTime | Last login timestamp |

Table 5: users collection schema

**Indexes:**

- `email` (unique)
- `role`

### 9.1.2 course_rooms Collection

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary key |
| name | String | Course name (3-100 chars) |
| description | String | Course description (0-500 chars) |
| invitation_code | String | 8-char unique code (indexed) |
| created_by | ObjectId | Faculty user reference |
| status | Enum | ACTIVE or ARCHIVED |
| created_at | DateTime | Creation timestamp |

Table 6: course_rooms collection schema

**Indexes:**

- `invitation_code` (unique)
- `(created_by, status)`

### 9.1.3 enrollments Collection

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary key |
| user_id | ObjectId | Student reference |
| course_id | ObjectId | Course reference |
| joined_at | DateTime | Enrollment timestamp |

Table 7: enrollments collection schema

**Indexes:**

- `(user_id, course_id)` (unique compound)
- `course_id`

### 9.1.4 videos Collection

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary key |
| course_id | ObjectId | Course reference |
| title | String | Video title |
| storage_url | String | Google Drive file ID |
| status | Enum | PENDING/PROCESSING/COMPLETE/FAILED |
| published | Boolean | Student visibility flag |
| duration_seconds | Integer | Video length in seconds |
| uploaded_at | DateTime | Upload timestamp |
| processed_at | DateTime | Processing completion time |

Table 8: videos collection schema

**Indexes:**

- `(course_id, published, status)`
- `status`

### 9.1.5 transcripts Collection

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary key |
| video_id | ObjectId | Video reference (1:1) |
| segments | Array | {start, end, text} objects |
| word_count | Integer | Total words |
| language | String | Detected language code |
| confidence | Float | Average confidence score |

Table 9: transcripts collection schema

**Indexes:** `video_id` (unique)

### 9.1.6 summaries Collection

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary key |
| video_id | ObjectId | Video reference |
| length_type | Enum | BRIEF/DETAILED/COMPREHENSIVE |
| content | String | Markdown-formatted summary |
| word_count | Integer | Summary length |
| version | Integer | Version number |
| is_published | Boolean | Visibility flag |
| created_at | DateTime | Creation timestamp |

Table 10: summaries collection schema

**Indexes:** `(video_id, is_published)`

### 9.1.7 quizzes Collection

| Field | Type | Description |
|---|---|---|
| _id | ObjectId | Primary key |
| video_id | ObjectId | Video reference |
| title | String | Quiz title |
| questions | Array | Question objects |
| is_published | Boolean | Visibility flag |
| version | Integer | Version number |
| created_at | DateTime | Creation timestamp |

Table 11: quizzes collection schema

**Question Schema:**

```
{
  "index": 0,
  "type": "MCQ",  // or "SHORT_ANSWER"
  "question": "What is a CNN?",
  "options": ["A", "B", "C", "D"],  // for MCQ only
  "correctAnswer": "2",  // index or text
  "explanation": "CNNs are...",
  "difficulty": "intermediate",
  "timestamp": 1234  // reference to video
}
```

Listing 7: Question Object Structure

## 9.2 Data Relationships

**Key Relationships:**

- User (Faculty) → CourseRooms (1:N)
- User (Student)  CourseRooms (M:N via enrollments)
- CourseRoom → Videos (1:N)
- Video → Transcript (1:1)
- Video → Summaries (1:N)
- Video → Quizzes (1:N)

**Referential Integrity:**

- Cascade delete: Course deletion removes all videos, enrollments
- Cascade delete: Video deletion removes transcript, summaries, quizzes
- Prevent delete: Cannot delete user with active course ownership

## 9.3 Data Access Patterns

**High-Frequency Queries:**

1. Get published videos for enrolled course
2. Get transcript for video playback
3. Check user enrollment (authorization)
4. Get video processing status

**Optimization Strategies:**

- Compound indexes on frequently queried combinations
- Field projection (fetch only needed fields)
- Redis caching with 5-minute TTL for read-heavy data
- Read from MongoDB secondaries for non-critical queries
- Pre-aggregated statistics (quiz averages, video counts)

# 10 Size and Performance

## 10.1 Expected Volumes

**Initial Deployment (Year 1):**

- Students: 100-500
- Faculty: 10-50
- Courses: 20-50
- Videos: 200-1,500
- Concurrent users: 50-100

**Growth Projection (Year 3):**

- Total users: 5,000-10,000
- Videos: 10,000-50,000
- Concurrent users: 500-1,000
- Daily active users: 1,000-2,000

**Storage Estimates:**

- Video files: 500 GB Year 1, 5 TB Year 3 (Google Drive)
- Database (Year 1): $< 5$ GB
- Database (Year 3): $< 50$ GB
- Redis cache: 2-4 GB

## 10.2 Performance Optimization

**Database Optimization:**

- Strategic indexing on frequent query patterns
- Query projection (fetch only required fields)
- Pagination for large result sets (20 items/page)
- Pre-aggregated statistics updated via background jobs

**Caching Strategy (Redis):**

- Session data: 4-hour TTL

- Course lists: 5-minute TTL
- Video metadata: 5-minute TTL
- Computation results: 24-hour TTL
- Cache invalidation on updates

**Asynchronous Processing:**

- Video processing in background workers
- Embedding generation offline
- Non-blocking API responses with status polling
- Email notifications asynchronous

## 10.3 Load Testing Scenarios

**Normal Load Test:**

- Users: 50 concurrent
- Duration: 30 minutes
- Success criteria: 95%ile response time $< 2$s
- Error rate: $< 0.1\%$

**Peak Load Test:**

- Users: 100 concurrent
- Duration: 15 minutes
- Success criteria: 95%ile response time $< 5$s
- Error rate: $< 1\%$

**Stress Test:**

- Ramp up to 200 users over 10 minutes
- Identify breaking point and degradation patterns
- Measure recovery time after load reduction

**Key Metrics:**

- Response time (50%ile, 95%ile, 99%ile)
- Requests per second (RPS)
- Error rate by endpoint
- CPU and memory utilization
- Database query performance

# 11 Quality Attributes

## 11.1 Scalability

**Horizontal Scaling:**

- Stateless API servers replicate behind load balancer
- Worker pool scales based on queue depth
- MongoDB sharding by `course_id` for multi-institution deployment
- Redis cluster scales with data volume

**Scaling Triggers:**

- API servers: CPU > 70% for 5 minutes
- Workers: Queue depth > 10 tasks
- Database: Storage > 80% capacity

## 11.2 Reliability

**Target:** 99.5% uptime during operational hours (8 AM - 8 PM, 7 days/week)

**Failure Recovery Mechanisms:**

- MongoDB replica set with automatic failover (30-60s)
- Retry logic with exponential backoff (3 attempts)
- Circuit breaker for external services (open after 5 failures)
- Health checks remove unhealthy nodes from pool
- Graceful degradation when AI services unavailable

**Data Integrity:**

- ACID transactions for critical operations
- Daily automated backups with 30-day retention
- Point-in-time recovery capability
- Audit logs for all data modifications

## 11.3 Security

**Authentication:**

- JWT with HMAC-SHA256 signature
- 4-hour token expiration
- Refresh token rotation
- Bcrypt password hashing (cost factor: 12)

**Authorization:**

- RBAC: Faculty and Student roles
- Course-scoped permissions
- Middleware enforces access controls
- Enrollment verification on every request

**Data Protection:**

- TLS 1.3 for all communications
- Encrypted database connections
- No plaintext passwords stored
- PII minimization (no SSN, credit cards)

**Compliance:**

- FERPA educational data privacy
- GDPR data portability and deletion
- 90-day audit log retention
- User consent for data processing

## 11.4 Maintainability

**Design Principles:**

- Layered architecture with separation of concerns
- Dependency injection for loose coupling
- Repository pattern abstracts data access
- Service layer encapsulates business logic

**Code Quality:**

- 80%+ test coverage requirement
- Automated linting (Pylint, ESLint)
- Type hints in Python code
- Comprehensive API documentation (OpenAPI/Swagger)

**Monitoring and Logging:**

- Centralized logging with structured format
- Error tracking with stack traces
- Performance metrics collection
- Alert system for critical failures

## 11.5 Usability

**Target:** System Usability Scale (SUS) score > 70

**User Experience Features:**

- Conversational AI reduces cognitive load
- Real-time progress indicators for processing
- Clear, actionable error messages
- Responsive design (desktop/tablet/mobile)
- Consistent UI patterns across features

**Accessibility:**

- WCAG 2.1 AA compliance
- Screen reader support
- Keyboard navigation
- High contrast color schemes
- Captions for all video content

# 12 Design Rationale and Decisions

## 12.1 Decision 1: MongoDB vs PostgreSQL

**Decision:** Use MongoDB as primary database

**Rationale:**

- Document model fits nested structures (video → transcript → segments)
- Schema flexibility for evolving AI content formats
- Natural sharding by `course_id` for multi-tenancy
- Simplified JSON serialization for REST API
- Horizontal scaling path for growth

**Trade-offs:**

- *Con*: Less mature ACID transactions than PostgreSQL
- *Mitigation*: MongoDB 4.0+ supports multi-document transactions
- *Con*: Complex joins require aggregation pipeline
- *Mitigation*: Acceptable for our query patterns (mostly document-centric)

## 12.2 Decision 2: Conversational AI Interface

**Decision:** Chat-based interface for faculty content generation

**Rationale:**

- Lower cognitive load vs complex form interfaces
- Enables iterative refinement ("Make it shorter")
- Contextual understanding from conversation history
- Natural language is intuitive for non-technical users
- Future-proof for voice input integration

**Trade-offs:**

- *Con*: Higher LLM API costs per interaction
- *Mitigation*: Caching, session management, rate limiting
- *Con*: Intent parsing complexity
- *Mitigation*: Specialized IntentParser service handles this

## 12.3 Decision 3: Draft/Published Workflow

**Decision:** Manual faculty review before student visibility

**Rationale:**

- Quality assurance before student exposure
- Liability mitigation for AI-generated content errors
- Faculty maintains content authority and control
- Clear audit trail of approvals
- Aligns with FERPA data control requirements

**Trade-offs:**

- *Con*: Additional publishing step required
- *Mitigation*: One-click publish action, batch operations
- *Con*: Not real-time for students
- *Mitigation*: Acceptable for batch preparation workflow

## 12.4 Decision 4: HuggingFace API vs Local GPU

**Decision:** Use HuggingFace Inference API for AI models

**Rationale:**

- No GPU hardware acquisition or maintenance costs
- Managed model updates and optimizations
- Pay-per-use pricing aligns with growth trajectory
- Access to latest open-source models
- Lower operational complexity

**Trade-offs:**

- *Con*: External API dependency
- *Mitigation*: Graceful degradation, circuit breaker pattern
- *Con*: Data sent to third-party
- *Mitigation*: HuggingFace is GDPR-compliant, no PII in requests
- *Con*: May need migration at very high scale
- *Mitigation*: Acceptable tradeoff for initial deployment

## 12.5 Decision 5: Celery for Asynchronous Processing

**Decision:** Use Celery with Redis for task queue

**Rationale:**

- Mature, production-tested solution
- Native Python integration
- Supports distributed workers
- Built-in retry and error handling
- Task monitoring and inspection tools

**Alternative Considered:** RabbitMQ as broker

- *Why rejected*: Redis simpler for our use case, already using for caching

# Appendix A: Requirements Traceability

Table 12: SRS to Design Component Mapping

| SRS ID | Requirement | Design Components |
|---|---|---|
| VP.Upload | Accept video files up to 2GB | VideoUploadService, GoogleDriveClient, validation middleware |
| VP.ASR | Speech to text with >90% accuracy | WhisperASRService, TranscriptRepository, Celery worker |
| VP.Scene-Detection | Detect slide changes with >85% accuracy | SceneDetectionService, CLIP integration |
| VP.Progress | Display real-time processing progress | ProcessingStatusDTO, polling endpoint |
| VP.Storage | Store videos securely | Google Drive API, MongoDB metadata |
| CS.Generation | Generate structured summaries | SummarizationEngine, FLAN-T5 model |
| CS.Timestamps | Include timestamps in summaries | Summary schema, transcript linking |
| CS.Concepts | Extract domain terminology | EmbeddingService, NLP pipeline |
| CS.Lengths | Support multiple summary lengths | lengthType enum, template system |
| CS.Navigation | Enable concept-based navigation | Frontend video player, timestamp links |
| QA.RAG | Answer questions with >80% relevance | RAGEngine, EmbeddingService, vector search |
| QA.Citations | Provide citations with timestamps | Citation schema, response formatter |
| QA.Navigation | Navigate to referenced segments | Frontend router, video player API |
| QA.Followup | Handle follow-up questions | ChatSession, context management |
| QA.Types | Support factual and conceptual questions | IntentParser, multiple LLM prompts |
| QG.Generation | Create MCQ and short-answer questions | QuizGenerator, question templates |
| QG.Rationales | Provide answer explanations | Quiz schema, explanation field |
| QG.Difficulty | Assign difficulty levels | Difficulty classifier, metadata |
| QG.Customization | Allow question customization | QuizOptionsDTO, filter logic |

| **SRS ID** | **Requirement** | **Design Components** |
|---|---|---|
| QG.Export | Support quiz export | Export service, format converters |
| KG.Generation | Create knowledge graphs | Knowledge graph service (future) |
| KG.Research | Integrate research papers | arXiv API client (future) |
| PERF.Processing | Process within 5 min/hr video | Celery workers, parallel processing |
| PERF.Response | API response <2s | Caching, indexing, connection pooling |
| SEC.Auth | Multi-factor authentication | JWT service, Bcrypt hashing |
| SEC.RBAC | Role-based access control | Authorization middleware, role enum |
| SEC.FERPA | FERPA compliance | Data isolation, audit logging, consent |
| SEC.Encryption | Encrypt data in transit and at rest | TLS 1.3, MongoDB encryption |
| AVAIL.Uptime | 99.5% uptime | Replica set, redundant servers, health checks |
| USA.Interface | SUS score >70 | React UI, usability testing, feedback loops |

# Appendix B: Glossary

**Aggregate Root** Domain-driven design pattern where an entity serves as the entry point for a cluster of related objects, controlling all access and ensuring consistency.

**ASR** Automatic Speech Recognition - technology that converts spoken audio into written text with timestamp alignment.

**Celery** Distributed task queue system for Python that enables asynchronous execution of long-running background jobs.

**CLIP** Contrastive Language-Image Pre-training - OpenAI's vision model that understands relationships between images and text, used for scene detection.

**Course Room** Isolated workspace where faculty manage content and students access materials, enforcing multi-tenant data boundaries.

**Draft Content** AI-generated materials visible only to faculty for review and refinement before publication to students.

**Embedding** Vector representation of text that captures semantic meaning in a high-dimensional space, enabling similarity search.

**FERPA** Family Educational Rights and Privacy Act - US federal law protecting student educational records and privacy.

**FLAN-T5** Fine-tuned Language Net Text-To-Text Transfer Transformer - Google's instruction-following language model for text generation tasks.

**Invitation Code** Unique 8-character alphanumeric code (excluding ambiguous characters) used for course enrollment.

**JWT** JSON Web Token - compact, URL-safe token format for securely transmitting authentication claims between parties.

**MongoDB** Document-oriented NoSQL database that stores data in flexible JSON-like documents.

**Multi-tenancy** Architectural approach where a single application instance serves multiple isolated customers (courses) with strict data separation.

**Published Content** Faculty-approved materials marked visible to enrolled students after quality review.

**RAG** Retrieval-Augmented Generation - AI technique that retrieves relevant context before generating responses, improving accuracy and enabling citations.

**RBAC** Role-Based Access Control - security model that restricts system access based on user roles (Faculty vs Student).

**Redis** In-memory data structure store used for caching, session management, and message brokering.

**Replica Set** MongoDB configuration with multiple database copies (primary + secondaries) for high availability and automatic failover.

**Repository Pattern** Design pattern that abstracts data access logic, providing a collection-like interface for domain objects.

**Whisper** OpenAI's state-of-the-art speech recognition model, multilingual and highly accurate for transcription tasks.

# Appendix C: Technology Stack

| Component | Technology | Version |
|---|---|---|
| **Frontend** | | |
| UI Framework | React | 18.2 |
| Styling | Tailwind CSS | 3.3 |
| HTTP Client | Axios | 1.4 |
| Routing | React Router | 6.14 |
| State Management | React Hooks | Built-in |
| **Backend** | | |
| Language | Python | 3.11 |
| Web Framework | FastAPI | 0.104 |
| ASGI Server | Uvicorn | 0.23 |
| Validation | Pydantic | 2.4 |
| Task Queue | Celery | 5.3 |
| Password Hashing | Bcrypt | 4.0 |
| JWT Library | PyJWT | 2.8 |
| **AI/ML Models** | | |
| Platform | HuggingFace API | Latest |
| ASR | Whisper | v3 |
| Summarization | FLAN-T5 | Large |
| Embeddings | sentence-transformers | all-MiniLM-L6-v2 |
| Vision | CLIP | vit-base-patch32 |
| **Data Storage** | | |
| Database | MongoDB | 7.0 |
| Cache/Queue | Redis | 7.2 |
| File Storage | Google Drive API | v3 |
| **Infrastructure** | | |
| Containers | Docker | 24.0 |
| Container Orchestration | Docker Compose | 2.20 |
| Load Balancer | Nginx | 1.24 |
| SSL Certificates | Let's Encrypt | Latest |
| **Development Tools** | | |
| Testing (Python) | pytest | 7.4 |
| Testing (JavaScript) | Jest | 29.6 |
| Linting (Python) | Pylint | 2.17 |
| Linting (JavaScript) | ESLint | 8.45 |
| API Documentation | Swagger/OpenAPI | 3.0 |

Table 13: Complete technology stack with versions

# Appendix D: API Security Specifications

## Authentication Flow

1. User submits credentials to `POST /api/v1/auth/login`
2. Server validates credentials against hashed password
3. Server generates JWT with 4-hour expiration
4. Server returns token to client
5. Client includes token in Authorization header: `Bearer <token>`
6. Server validates token on each request via middleware

## JWT Token Structure

```
{
  "sub": "507f1f77bcf86cd799439011",  // user_id
  "email": "faculty@university.edu",
  "role": "FACULTY",
  "iat": 1696608000,  // issued at (unix timestamp)
  "exp": 1696622400   // expires at (unix timestamp)
}
```

Listing 8: JWT Payload Example

## Rate Limiting

| Endpoint Category | Rate Limit |
|---|---|
| Authentication | 5 requests/min per IP |
| Video Upload | 10 requests/hour per user |
| AI Content Generation | 10 requests/min per user |
| General API | 100 requests/min per user |

Table 14: Rate limiting by endpoint

## Input Validation

All API inputs validated using Pydantic schemas:

- Email: RFC 5322 format validation
- Password: Min 8 chars, uppercase, lowercase, number, special char
- Course name: 3-100 characters

- Video file: MP4/AVI/MOV, max 2GB
- Message text: 1-1000 characters

**End of Document**